Réponses au questionnaire pour le livrable 2

Bongiovanni Arthur, Vanicotte Alexandre, Withanage-perera Sakun, Joseph Wilkens

1. Comment les commandes de mouvement sont-elles envoyées au drone ?

Les commandes de mouvement sont transmises au drone via la classe **DroneController**, qui implémente les interfaces **ARDeviceControllerListener** et **ARDeviceControllerStreamListener** du SDK Parrot AR. Ces interfaces définissent les méthodes permettant de contrôler les opérations du drone.
Les commandes sont déclenchées par des interactions utilisateur dans la classe **GUIGame**, où des écouteurs sont attachés aux boutons affichés à l'écran (par exemple, les contrôles directionnels). Ainsi, l'appui sur un bouton « tourner à gauche » déclenche une rotation vers la gauche tant que le bouton reste enfoncé.
De plus, le gyroscope de l'appareil détecte les changements d'orientation, ce qui permet des ajustements directionnels basés sur l'inclinaison pour un contrôle plus intuitif.

2. En mode multijoueur, quelles données les smartphones échangent-ils pendant la course ?

Lors des courses multijoueurs, les smartphones échangent différentes données de synchronisation et d'état du jeu via une communication Bluetooth, assurée par le module **BluetoothCommunication** et gérée à travers les callbacks **BluetoothCommunicationListener** dans la classe **Game**. Les principaux échanges incluent :

- **Signaux de disponibilité** : un statut « prêt » est envoyé lorsqu'un joueur est prêt, déclenchant `onSecondPlayerReady()` pour initialiser le drone adverse.
- **Données du circuit** : le serveur transmet les détails de configuration du circuit via `comBT.sendCircuit()`, reçus par le client avec `onCircuitReceived()`.
- **Démarrage de la course** : un signal de départ est diffusé, invoquant `onSecondStartRace()` afin de synchroniser le lancement de la course.
- **Mises à jour de progression** : les tours complétés sont communiqués via `onSecondPlayerLapFinished()`, mettant à jour le compteur de tours de l'adversaire.

- **Utilisation des objets** : des messages textuels sont échangés pour les effets des objets, formatés comme `"type[/SYMBOLE]"` (par exemple `"redshell"` pour des effets instantanés, `"banana/<SYMBOLE>"` pour des objets placés), traités par `onSecondPlayerUsesItem()`.
- **Interactions avec les objets** : lorsqu'un objet est collecté, un message contenant le symbole associé est envoyé, déclenchant `onSecondPlayerTouchedItem()` afin de le supprimer localement.
- **Fin de partie** : des messages indiquent la fin de la course (`onSecondPlayerFinished()`) ou l'abandon (`onSecondPlayerGaveUp()`), entraînant l'affichage de notifications de victoire/défaite et le nettoyage des ressources.

3. Comment les objets sont-ils placés sur le circuit dans l'application Android ? En mode solo, peut-on placer un objet sur le circuit ? Justifiez.

Les objets sont placés sur le circuit lorsqu'un joueur utilise un objet, ce qui est détecté par la reconnaissance de symboles AR. La classe **GUIGame** invoque `Game.onItemUsed()`, qui notifie les écouteurs enregistrés (dont **GUIGame** pour l'affichage local et **BluetoothCommunication** pour la synchronisation multijoueur).
Pour les objets placés (par exemple **Banana** ou **FakeBox**), l'instance **Circuit** ajoute l'objet via `addObject()`, et le moteur de rendu l'affiche à l'emplacement du symbole spécifié à l'aide de `defineModelAtSymbol()`.

En mode solo, les objets peuvent effectivement être placés sur le circuit. Même sans communication Bluetooth (`comBT == null`), l'interface **GUIGame** est toujours enregistrée comme écouteur dans le constructeur de **Game**, ce qui garantit que le placement et l'affichage local de l'objet ont lieu lors de son utilisation. Aucune transmission réseau n'est nécessaire puisqu'il n'y a pas d'adversaire à synchroniser.

4. À quels moments ces objets sont-ils placés ?

Les objets sont placés au moment exact de l'utilisation de l'objet, que ce soit localement ou à distance :

- **Utilisation locale** : lorsqu'un joueur active un objet (détecté via `DetectionTask.Symbol`), **GUIGame** déclenche immédiatement le placement via `Game.onItemUsed()`, ajoutant l'objet au **Circuit** et l'affichant localement.
- **Utilisation distante (multijoueur)** : à la réception d'un message Bluetooth (par exemple `"banana/<SYMBOLE>"`), `Game.onSecondPlayerUsesItem()` décode le

message, crée l'objet, l'ajoute au **Circuit** et l'affiche via `guiGame.getRenderer().defineModelAtSymbol()`.

Remarque sur le timing : le placement est instantané dès la détection ou la réception du message, sans délai. En mode solo, il se produit uniquement lors de l'utilisation locale, sans surcharge Bluetooth.

5. Comment l'application détecte-t-elle un objet sur le circuit ?

La détection des objets repose sur la reconnaissance de marqueurs AR à l'aide de la classe **DetectionTask**, qui analyse le flux caméra pour identifier les marqueurs `DetectionTask.Symbol`. Lorsqu'un symbole est détecté, **GUIGame** appelle `GUIGameListener.onSymbolTouched()`, ce qui invoque `Game.onSymbolTouched()`. Cette méthode récupère l'objet associé via `Circuit.getInstance().getObjects().get(symbol)`, notifie les écouteurs avec `onItemTouched()`, supprime l'objet du circuit à l'aide de `removeObject()`, et met à jour l'affichage via `deleteModelAtSymbol()`. En mode multijoueur, la suppression est également communiquée via Bluetooth afin de synchroniser la vue de l'adversaire.

6. Peut-on éviter un objet sur le circuit ? Justifiez.

Oui, mais l'évitement dépend du type d'objet :

- **Objets placés** (par exemple **Banana**, **FakeBox**) : ils peuvent être évités en dirigeant le drone pour éviter les marqueurs AR associés. La détection et l'activation de l'effet nécessitent que la caméra reconnaisse le symbole `DetectionTask.Symbol`, comme implémenté dans `Game.onSymbolTouched()`, qui ne se déclenche qu'au contact du marqueur.
- **Objets à effet instantané** (par exemple **RedShell**, **Blooper**) : ils ne peuvent pas être évités, car leurs effets sont appliqués immédiatement à la réception du message Bluetooth dans `Game.onSecondPlayerUsesItem()`, sans nécessiter de proximité physique ni de détection AR.

Cette distinction garantit un gameplay stratégique, où les objets placés récompensent l'évitement, tandis que les effets instantanés introduisent une part d'imprévisibilité.

7. Comment la fin d'un tour est-elle détectée ? Et

l’arrivée ?

La détection des tours et de l’arrivée repose sur la reconnaissance de marqueurs AR via **DetectionTask**, avec des callbacks relayés de **GUIGame** vers **Game** :

- **Fin de tour** : la détection du marqueur de ligne d’arrivée déclenche `Game.onPlayerDetectsArrivalLine()`. Le système vérifie d’abord que les checkpoints requis ont été franchis (`drone.getCurrentCheckpoint() >= Circuit.getInstance().getCheckpointToCheck()`). Si c’est le cas, le compteur de tours est incrémenté (dans la limite du nombre total de tours), les checkpoints sont réinitialisés à zéro, et les écouteurs sont notifiés via `onPlayerFinishedLap()`. Si le nombre de tours atteint le total du circuit, la course se termine avec `onPlayerFinished()`.
- **Checkpoints et arrivée** : les marqueurs de checkpoint invoquent `Game.onPlayerDetectsCheckpoint()`, incrémentant le compteur de checkpoints. La ligne d’arrivée ne valide un tour que si les checkpoints sont suffisants, garantissant une progression correcte.

Ce système basé sur la réalité augmentée impose le respect du circuit, avec une synchronisation Bluetooth en mode multijoueur.

8. Comment les marqueurs sont-ils liés aux objets placés dans l’application ?

Les marqueurs sont liés aux objets via le mécanisme de correspondance interne de la classe **Circuit**. Chaque marqueur est représenté par une valeur de l’énumération `DetectionTask.Symbol`, utilisée comme clé dans une map (`Circuit.getInstance().getObjects()`). Lorsqu’un objet est placé :

- `Circuit.addObject(symbol, item)` associe le symbole à l’objet.
- Le moteur de rendu affiche l’objet à la position du marqueur via `defineModelAtSymbol(item, symbol)`.
- Lors de la détection, `Game.onSymbolTouched(symbol)` récupère l’objet avec `getObjects().get(symbol)`, gère l’interaction, puis supprime l’association avec `removeObject(symbol)`.

En mode multijoueur, les messages incluent le symbole (par exemple `"banana/<SYMBOLE>"`) afin de reproduire le lien sur l’appareil adverse et garantir un positionnement AR cohérent.

9. En mode multijoueur, comment le circuit est-il initialisé pour les deux joueurs ?

L'initialisation du circuit est gérée de manière asymétrique afin de synchroniser les deux joueurs :

- Le serveur (désigné par `isServer == true` dans le constructeur de **Game**) possède l'instance initiale du circuit (`Circuit.initInstance()`) et transmet ses données via `comBT.sendCircuit()` .
- Le client reçoit le circuit via **BluetoothCommunication**, qui configure l'instance locale de **Circuit** et notifie **Game** avec `onCircuitReceived()` .
- `Game.onCircuitReceived()` déclenche des vérifications de disponibilité, permettant le démarrage de la course une fois toutes les conditions réunies (flux vidéo, contrôleur du drone, disponibilité des joueurs, etc.).

Cela garantit que les deux joueurs utilisent une configuration de circuit identique, gérée par le singleton partagé `Circuit.getInstance()` pour les checkpoints, les tours et les objets.